



WOLLO University

Department of software engineering

Blueprint based exit material for

Software & information security

Prepared by:

@Neyatibeb

KOMBOLCHA, Ethiopia

2015 E.C

❖ Principle method and technology to make software more secure

● Principle

- Defense in depth: This principle states that security should be layered, with multiple layers of protection in place. This way, if one layer is compromised, the others can still provide some level of protection. For example, implementing both network-level and application-level firewalls, or using both access control and encryption to protect sensitive data
- Separation of duties: This principle involves dividing responsibilities so that no single person or process has too much control over a system. For example, separating development and production environments, or separating the roles of system administrator and database administrator.
- Secure by design: This principle involves designing software systems with security in mind from the outset, rather than attempting to retrofit security controls onto existing software. This can involve using security design patterns, threat modeling, and code reviews to identify and mitigate potential vulnerabilities.
- Fail-safe defaults: This principle states that software should be configured in a secure way by default. This means that users should not have to take any special steps to secure the software, as it will already be secure by default.
- Least privilege: This principle states that users should only have the privileges they need to perform their job. This helps to reduce the risk of unauthorized access to sensitive data.
- Least functionality: This principle states that software should only have the functionality that is necessary. This helps to reduce the attack surface of the software, making it less vulnerable to attack.
- Simplification: This principle involves minimizing the complexity of software systems to reduce the likelihood of security vulnerabilities. Simple systems are easier to understand, test, and secure than complex systems.

● Method

- Static analysis: This method involves analyzing the source code of software to identify potential security vulnerabilities.
- Dynamic analysis: This method involves executing software in a controlled environment to identify potential security vulnerabilities.
- Penetration testing: This method involves hiring a third-party security firm to attempt to hack into the software. This can help to identify security vulnerabilities that may have been missed by other methods.

● Technology

- Intrusion detection systems: These systems monitor network traffic for signs of malicious activity.
- Firewalls: These systems control access to a network, preventing unauthorized users from accessing sensitive data.
- Antivirus software: This software scans files for known viruses and other malware.
- Vulnerability scanning tools: These tools scan software for potential security vulnerabilities.

❖ Threat and vulnerability that make software less secure and how avoid them

- Some of the typical threats and vulnerabilities that make software less secure, and how to avoid them:
 - Bugs: Bugs are errors in software code that can lead to security vulnerabilities. They can be caused by a variety of factors, such as human error, incorrect assumptions, or incomplete testing. To avoid bugs, software developers should follow coding best practices, use static analysis tools, and conduct thorough testing.
 - Exposure of sensitive data: Sensitive data, such as passwords, credit card numbers, and social security numbers, should be properly protected. This means storing it in a secure location, encrypting it, and only accessing it when necessary. To avoid exposing sensitive data, organizations should implement strong data security policies and procedures.
 - Flaws in injection: Injection flaws occur when untrusted data is inserted into a software application in a way that can be used to execute malicious code. This can happen through input fields, parameters, or headers. To avoid injection flaws, organizations should use input validation techniques to filter out malicious data.
 - Buffer overflows: Buffer overflows occur when too much data is written to a buffer, which can cause the software to crash or execute malicious code. To avoid buffer overflows, organizations should use secure coding practices and ensure that buffers are sized appropriately.
 - Security misconfiguration: Security misconfigurations are errors in the configuration of software or systems that can leave them vulnerable to attack. This can include things like leaving default passwords in place, enabling unnecessary services, or opening up ports that are not needed. To avoid security misconfigurations, organizations should have a process for regularly reviewing and updating their security configurations.
- Here are some additional tips for avoiding software vulnerabilities:
 - Use secure development practices, such as OWASP's Top 10.
 - Implement security testing into the software development lifecycle.
 - Keep software up to date with the latest security patches.
 - Use a vulnerability scanner to identify and fix security issues.
 - Educate employees about security risks and how to protect themselves

❖ Prevent threat, mitigate their effect by improving the software design and implementation

- some ways to prevent threats and mitigate their effects by improving the software's design and implementation:
 - Use secure design principles. This includes principles such as least privilege, defense in depth, and fail-safe defaults.
 - Use secure coding practices. This includes practices such as input validation, output encoding, and secure session management.
 - Implement security testing throughout the software development lifecycle. This includes unit testing, integration testing, and system testing.
 - Use a security framework. A security framework can provide a set of security controls that can be used to protect software.

Department of software engineering

- Keep software up to date with the latest security patches. Software vendors often release security patches to fix vulnerabilities.
 - Use a vulnerability scanner to identify and fix security issues. A vulnerability scanner can scan software for known vulnerabilities.
 - Educate employees about security risks and how to protect them. Employees should be aware of the security risks that they face and how to protect themselves.
- Here are some additional specific things that can be done to improve the software's design and implementation to prevent threats:
- Use modularity. This can make it easier to identify and fix security vulnerabilities.
 - Encapsulation. This can help to protect sensitive data from unauthorized access.
 - Abstraction. This can help to reduce the complexity of the software and make it easier to understand.
 - Logging and auditing. This can help to track down security incidents and identify the root cause.
 - Use of security controls. This includes things like firewalls, intrusion detection systems, and data encryption.

❖ Tool and technique for testing verifying that software secure

- There are a number of tools and techniques that can be used to test and verify that software is secure. Some of the most common include:
- Static application security testing (SAST): SAST tools scan the source code of software for potential security vulnerabilities. This can be done before the software is compiled, which can help to identify vulnerabilities early in the development process.
 - Dynamic application security testing (DAST): DAST tools run the software in a live environment and test it for vulnerabilities. This can help to identify vulnerabilities that are only exposed when the software is running.
 - Interactive application security testing (IAST): IAST tools combine SAST and DAST to provide a more comprehensive view of the software's security. This can help to identify vulnerabilities that may be missed by either SAST or DAST alone.
 - Penetration testing (PT): PT is a manual process where a security professional attempts to exploit vulnerabilities in the software. This can be a very effective way to identify vulnerabilities that are not easily detected by automated tools.
 - Vulnerability scanning: Vulnerability scanning tools can be used to identify known vulnerabilities in software. This can be a quick and easy way to identify vulnerabilities, but it is important to note that not all vulnerabilities will be detected by vulnerability scanners.
 - Security code reviews: Security code reviews are conducted by a team of security experts who review the source code of software for potential security vulnerabilities.
 - Threat modeling: Threat modeling is a process of identifying and mitigating potential threats to software. This can be done by identifying the assets that need to be protected, the threats that could impact those assets, and the controls that can be put in place to mitigate those threats.
 - Security awareness training: Security awareness training is essential for all employees who interact with software. This training should cover topics such as phishing, social engineering, and password security.

-